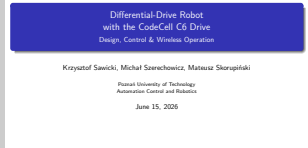
Differential-Drive Robot with the CodeCell C6 Drive

Design, Control & Wireless Operation

Krzysztof Sawicki, Michał Szerechowicz, Mateusz Skorupiński

Poznań University of Technology
Automation Control and Robotics

June 15, 2026



Introduce the project as a compact differential-drive robot built around the CodeCell C6 Drive board. Mention that the presentation covers the mechanical layout, electronics, control software, wireless interface, and safety mechanisms.

Outline

1. Project Overview
2. Hardware
3. Software Architecture
4. Movement Control
5. Wireless Communication
6. Control Application
7. Safety & Robustness
8. Summary & Future Work

Outline
Project Overview
Hardware
Software Architecture
Movement Control
Wireless Communication
Control Application
Safety & Robustness
Summary & Future Work

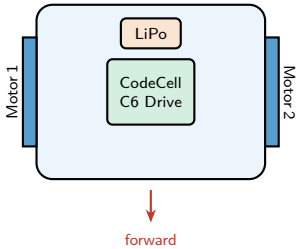
Briefly walk through the structure: project goals, hardware, mechanical design, software architecture, movement control, wireless communication, browser interface, safety, and future work. Keep this slide short and use it only as orientation.

Project Goals

Build a **compact, wirelessly controlled** differential-drive robot using the CodeCell C6 Drive as the sole electronic module.

Key objectives:

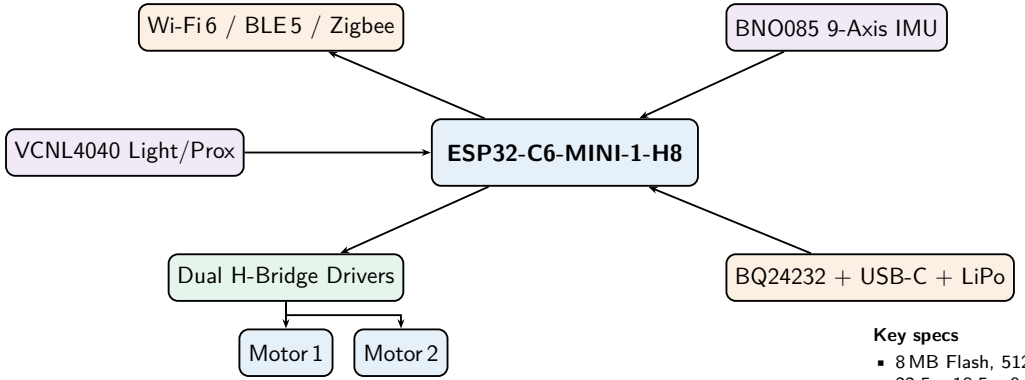
- Heading-stabilised motion via onboard IMU
- Two operating modes: real-time joystick & autonomous point-to-point
- Browser-based control app (no install needed)
- 3D-printed chassis – reproducible build



Top-down schematic view

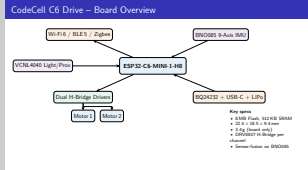
Explain that the main idea was to build a small mobile robot with minimal external electronics. Emphasize that the CodeCell C6 Drive acts as the core module: motor driver, wireless interface, and sensor platform. Point out the two modes: direct joystick control and autonomous target movement.

CodeCell C6 Drive – Board Overview



Key specs

- 8 MB Flash, 512 KB SRAM
- $22.5 \times 18.5 \times 9.4$ mm
- 3.4 g (board only)
- DRV8837 H-Bridge per channel
- Sensor-fusion on BNO085



Describe the board as the central hardware element. Highlight the ESP32-C6 for computation and wireless communication, the BNO085 IMU for heading estimation, the motor drivers for the two wheels, and the LiPo/USB-C power path. Mention that using one compact board simplifies wiring and makes the robot easier to reproduce.

Mechanical Design

Differential-drive layout

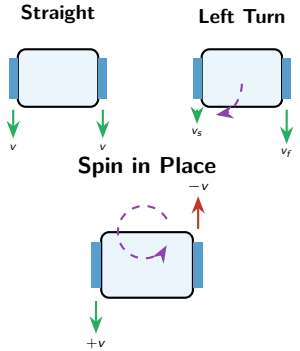
- Left motor → left wheel
- Right motor → right wheel
- Board and LiPo mounted centrally

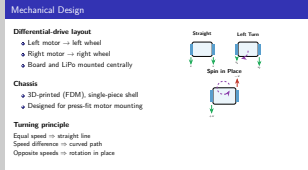
Chassis

- 3D-printed (FDM), single-piece shell
- Designed for press-fit motor mounting

Turning principle

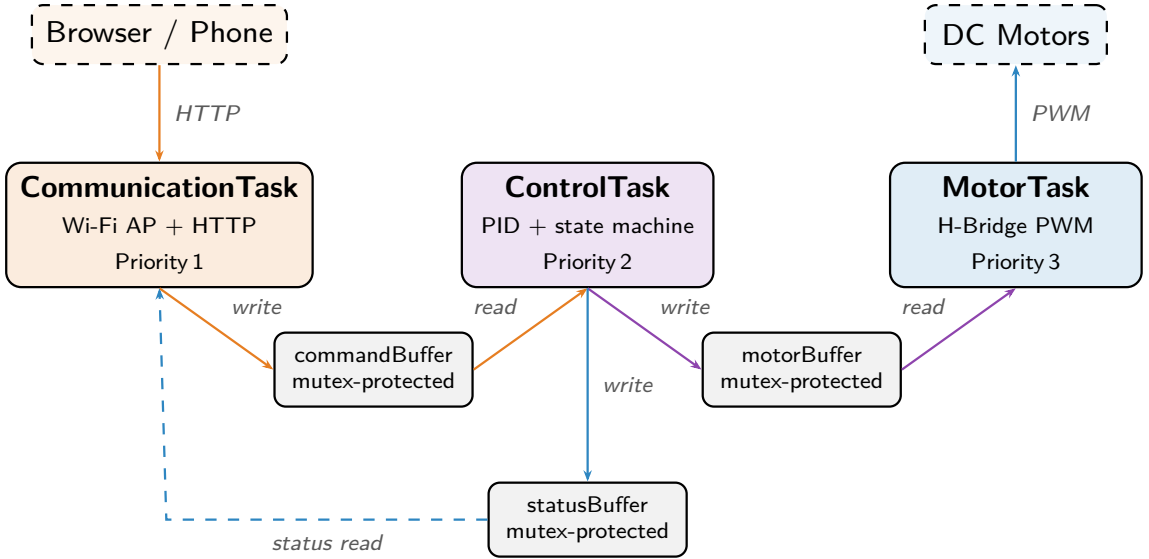
Equal speed ⇒ straight line
Speed difference ⇒ curved path
Opposite speeds ⇒ rotation in place

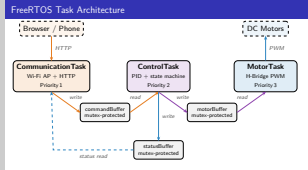




Explain the differential-drive principle: two independently driven wheels create straight motion, turns, and rotation in place. Mention that the chassis is 3D printed and designed to hold the board, battery, and motors compactly. Use the three diagrams to show equal speed, unequal speed, and opposite wheel directions.

FreeRTOS Task Architecture





Explain that the firmware is split into three FreeRTOS tasks. The communication task handles HTTP commands, the control task interprets commands and computes motor targets, and the motor task drives the H-bridges. Point out that shared buffers are protected by mutexes so tasks do not write over each other.

Differential-Drive Motor Mixing

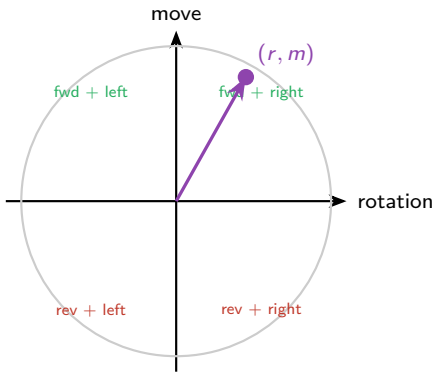
Joystick inputs in $[-100, 100]$:

- `move`: forward/backward
- `rotation`: turning

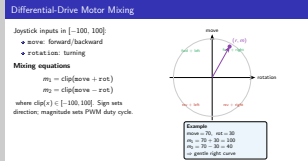
Mixing equations

$$m_1 = \text{clip}(\texttt{move} + \texttt{rot})$$
$$m_2 = \text{clip}(\texttt{move} - \texttt{rot})$$

where $\text{clip}(x) \in [-100, 100]$. Sign sets direction; magnitude sets PWM duty cycle.



Example
move $= 70$, rot $= 30$
$m_1 = 70 + 30 = 100$
$m_2 = 70 - 30 = 40$
$\Rightarrow$ gentle right curve



Describe how joystick input is converted into left and right motor commands. The forward/backward value is combined with the rotation value. A positive rotation increases one motor and decreases the other, creating a turn. The example shows a forward-right curve where one wheel moves faster than the other.

Heading-Stabilised Control – PID on Yaw

The BNO085 provides fused **yaw** (heading) at each control cycle (∼50 Hz). A discrete PID controller minimises heading error and feeds its output as the `rotation` input to the motor mixer.

Tuning parameters

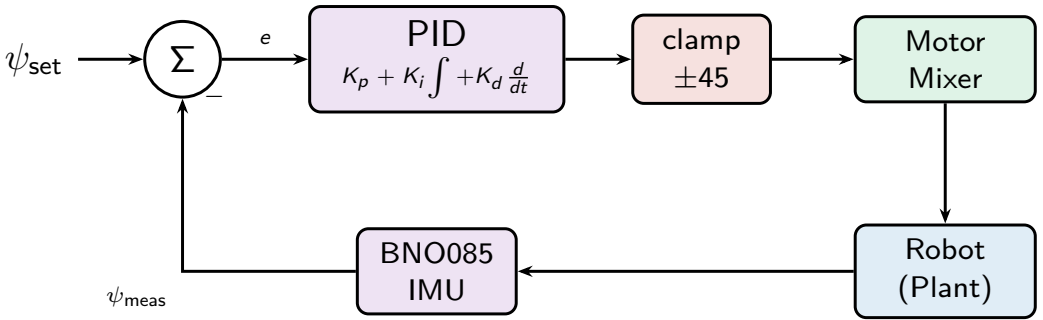
Parameter	Value
K_p	0.30
K_i	0.02
K_d	0.02
Integral limit	± 120
Max correction	± 45

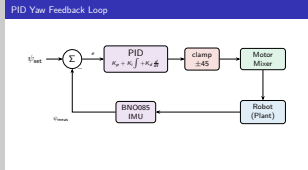
Error is wrapped to $(-180°, 180°]$ so the controller always uses the shortest rotation path.

Heading-Stabilised Control – PID on Yaw
The BNO085 provides fused **yaw** (heading) at each control cycle (∼50 Hz). A discrete PID controller minimises heading error and feeds its output as the `rotation` input to the motor mixer.
Tuning parameters
Parameter Value
K_p 0.30
K_i 0.02
K_d 0.02
Integral limit ± 120
Max correction ± 45
Error is wrapped to $(-180°, 180°]$ so the controller always uses the shortest rotation path.

Explain that the BNO085 gives a fused yaw angle, which is more useful than raw accelerometer or gyro data alone. The controller compares the desired heading with the measured heading. The PID constants were chosen conservatively so the robot corrects drift without oscillating strongly.

PID Yaw Feedback Loop





Walk through the feedback loop from left to right. The set heading is compared with measured heading, PID produces a correction, the correction is clamped, and the motor mixer applies it to the wheels. The robot motion changes the measured yaw, closing the loop through the IMU.

Control Modes – Direct vs. Target

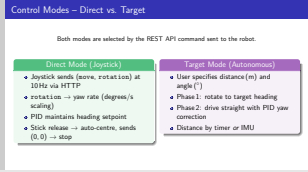
Both modes are selected by the REST API command sent to the robot.

Direct Mode (Joystick)

- Joystick sends (`move`, `rotation`) at 10 Hz via HTTP
- `rotation` → yaw rate (degrees/s scaling)
- PID maintains heading setpoint
- Stick release → auto-centre, sends $(0, 0)$ → stop

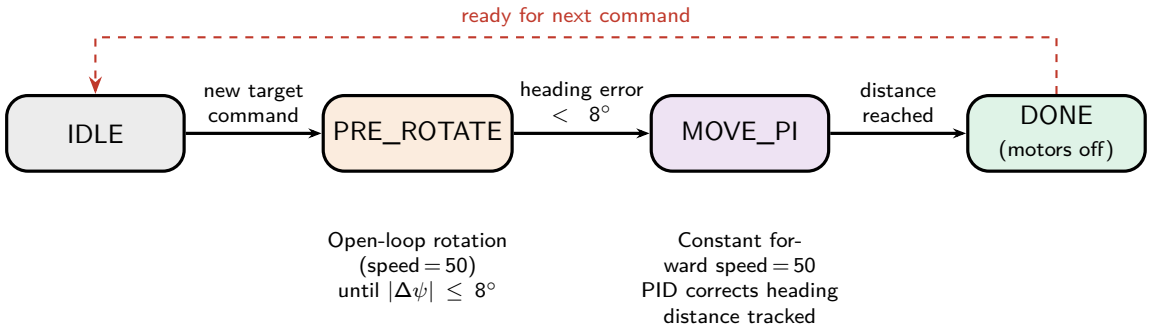
Target Mode (Autonomous)

- User specifies distance (m) and angle (°)
- Phase 1: rotate to target heading
- Phase 2: drive straight with PID yaw correction
- Distance by timer *or* IMU

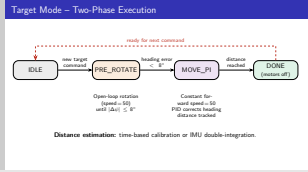


Compare the two control modes. Direct mode is for manual driving from the browser joystick and continuously sends movement commands. Target mode is more autonomous: the user gives a distance and angle, and the robot executes the motion in phases.

Target Mode – Two-Phase Execution



Distance estimation: time-based calibration or IMU double-integration.



Explain that target mode is implemented as a simple state machine. First, the robot rotates until the heading error is small enough. Then it drives forward while the PID keeps the heading stable. After the distance is reached, motors stop and the robot returns to idle.

Start Ramp & Motor Deadband Compensation

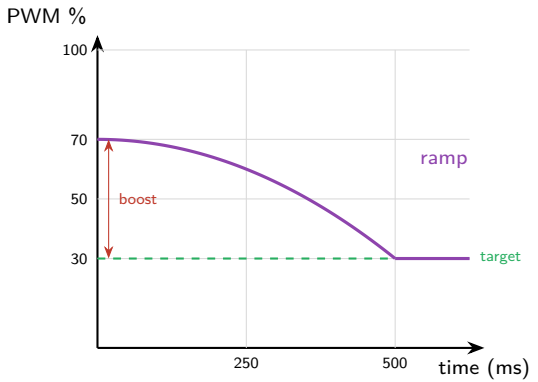
Small DC motors need extra torque to overcome **static friction** at startup.

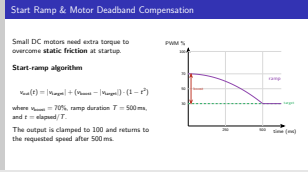
Start-ramp algorithm

$$v_{\text{out}}(t) = |v_{\text{target}}| + (v_{\text{boost}} - |v_{\text{target}}|) \cdot (1 - t^2)$$

where $v_{\text{boost}} = 70\%$, ramp duration $T = 500\,\text{ms}$, and $t = \text{elapsed}/T$.

The output is clamped to 100 and returns to the requested speed after 500 ms.





Explain why the start ramp is needed: small DC motors may not move reliably at low PWM because of static friction. The algorithm briefly applies a higher boost and then smoothly decays to the requested speed. This makes starts more reliable without keeping the motor overpowered.

Wi-Fi Access Point & REST API

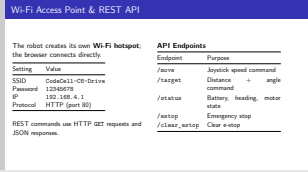
The robot creates its own **Wi-Fi hotspot**; the browser connects directly.

Setting	Value
SSID	`CodeCell-C6-Drive`
Password	`12345678`
IP	`192.168.4.1`
Protocol	HTTP (port 80)

REST commands use HTTP `GET` requests and JSON responses.

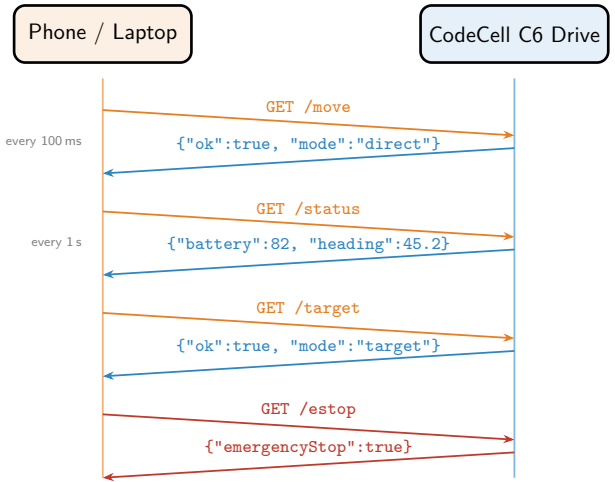
API Endpoints

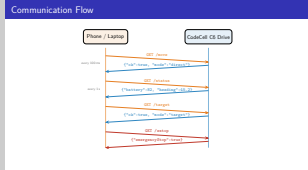
Endpoint	Purpose
`/move`	Joystick speed command
`/target`	Distance + angle command
`/status`	Battery, heading, motor state
`/estop`	Emergency stop
`/clear_estop`	Clear e-stop



Describe the communication setup. The robot creates its own Wi-Fi access point, so no external router is required. The browser sends simple HTTP GET requests to control movement, request status, and trigger or clear the emergency stop.

Communication Flow





Use this slide to explain the message sequence. The browser sends movement commands frequently, status is polled less often, target commands start autonomous motion, and emergency stop has a dedicated endpoint. Emphasize that the API is simple enough to test from any browser.

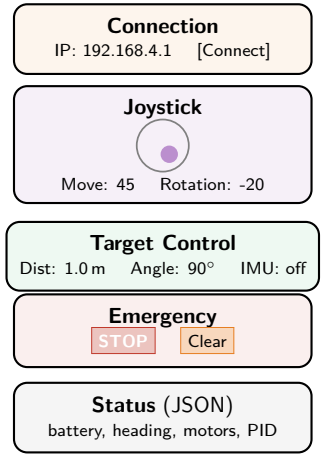
Web-Based Control Interface

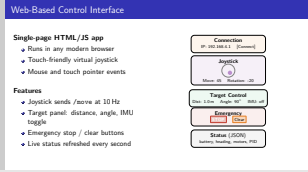
Single-page HTML/JS app

- Runs in any modern browser
- Touch-friendly virtual joystick
- Mouse and touch pointer events

Features

- Joystick sends `/move` at 10 Hz
- Target panel: distance, angle, IMU toggle
- Emergency stop / clear buttons
- Live status refreshed every second





Explain that the control application is a single-page web interface. It works on a phone or laptop and supports both mouse and touch input. Mention the main controls: connection, joystick, target command panel, emergency stop, and live status.

Safety Mechanisms

Emergency Stop

- One-click E-Stop via `/estop`
- All tasks check one shared flag
- Cleared only by explicit `/clear_estop`

Command Timeout

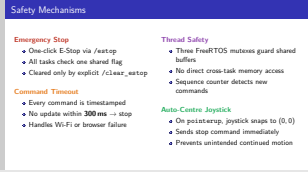
- Every command is timestamped
- No update within **300 ms** $\rightarrow$ stop
- Handles Wi-Fi or browser failure

Thread Safety

- Three FreeRTOS mutexes guard shared buffers
- No direct cross-task memory access
- Sequence counter detects new commands

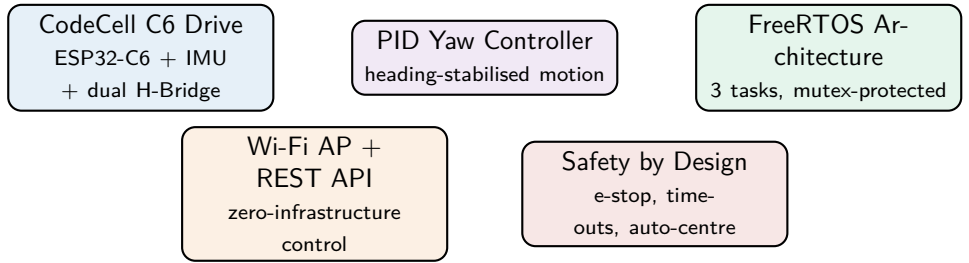
Auto-Centre Joystick

- On `pointerup`, joystick snaps to $(0, 0)$
- Sends stop command immediately
- Prevents unintended continued motion

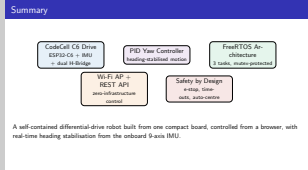


Present safety as a core part of the design. Emergency stop overrides normal motion, command timeout stops the robot if communication is lost, mutexes protect shared state, and joystick auto-centering prevents the robot from continuing to move after input is released.

Summary



A self-contained differential-drive robot built from one compact board, controlled from a browser, with real-time heading stabilisation from the onboard 9-axis IMU.



Summarize the project as a self-contained differential-drive robot built from a compact board with integrated wireless control, motor driving, and IMU feedback. Reinforce the main technical achievements: FreeRTOS architecture, PID heading control, browser control, and safety mechanisms.

Future Work

- **Obstacle avoidance** – reactive stop / turn behaviours
- **Odometry improvements** – wheel encoders or visual odometry
- **Multi-waypoint navigation** – sequence of target commands
- **BLE / Zigbee control** – lower-power communication options
- **Smartphone app** – richer UI and persistent settings

Future Work
- **Obstacle avoidance** – reactive stop / turn behaviours
- **Odometry improvements** – wheel encoders or visual odometry
- **Multi-waypoint navigation** – sequence of target commands
- **BLE / Zigbee control** – lower-power communication options
- **Smartphone app** – richer UI and persistent settings

Mention practical extensions. Obstacle avoidance could use the proximity sensor, odometry could improve distance estimation, multi-waypoint navigation would make target mode more useful, and BLE/Zigbee or a smartphone app could improve the user experience.

Thank you

Questions?

Source code & 3D models available in the project repository

Thank you
Questions?
Source code & 3D models available in the project repository

Close by inviting questions. Be ready to answer about PID tuning, communication reliability, mechanical design choices, and how the robot estimates distance in target mode.